

COMP4601: Team HOG Riders



COMP4601 Project: Accelerated Histograms of Gradients

Name	Email	zID
Samyak Diwan	z5611048@ad.unsw.edu.au	z5611048
Abhiram Yanamandra	z549219@ad.unsw.edu.au	z549219
Srikanan Gomadam	z5478391@ad.unsw.edu.au	z5478391
Rhythm Nath	z5417295@ad.unsw.edu.au	z5417295

Project overview: aims & results

Take HoG feature extraction off the CPU and prove the FPGA runs it correctly and dramatically faster on a real 10,000-image batch.



WHAT WE SET OUT TO DO

From a verified C++ baseline to an iteratively optimised HLS kernel

- ✓ Baseline: time the C/C++ HoG on the ARM PS (420 μ s / image)
- ✓ Port to Vitis HLS at 200 MHz
- ✓ Benchmark on 10,000 real images on the KV260 not simulation
- ✓ Then optimise in verified steps: every change passes csim $\rightarrow$ csynth $\rightarrow$ cosim $\rightarrow$ board before the next
- ✓ Report the resource cost of every speedup, not just the wins

INTERIM RESULT 6.16 $\times$ $\rightarrow$ FINAL

135.5 $\times$ kernel speedup

on the Kria KV260, measured across all 10,000 images on real hardware

ALSO DELIVERED

- ✓ 129.7 $\times$ end-to-end transfers now < 5% of runtime
- ✓ 308,724 images / s; the full batch finishes in 32.4 ms
- ✓ Correctness identical at every step: 9,998 / 10,000 within tolerance

System design: batched host, dataflow kernel

A clean split software owns batching, control and verification; hardware owns the entire per-image compute pipeline.

PROCESSING SYSTEM

ARM PS · PetaLinux

hog_host orchestration only

- Loads all 10,000 BMPs into one flat buffer
- ONE batched XRT launch for the whole set
- Times write / kernel / read phases

SHARED MEMORY

DDR / AXI

10.2 MB images in
11.5 MB features out
512-bit AXI-M bursts,
auto-widened

PROGRAMMABLE LOGIC

hog_compute @ 200 MHz · Vitis HLS 2025.2

Load

Normalise

Gradients

Histograms

Block Norm

Store

Six concurrent dataflow stages; ping-pong buffers between stages so consecutive images overlap.
One s_axilite scalar (num_images) + two 512-bit m_axi ports.

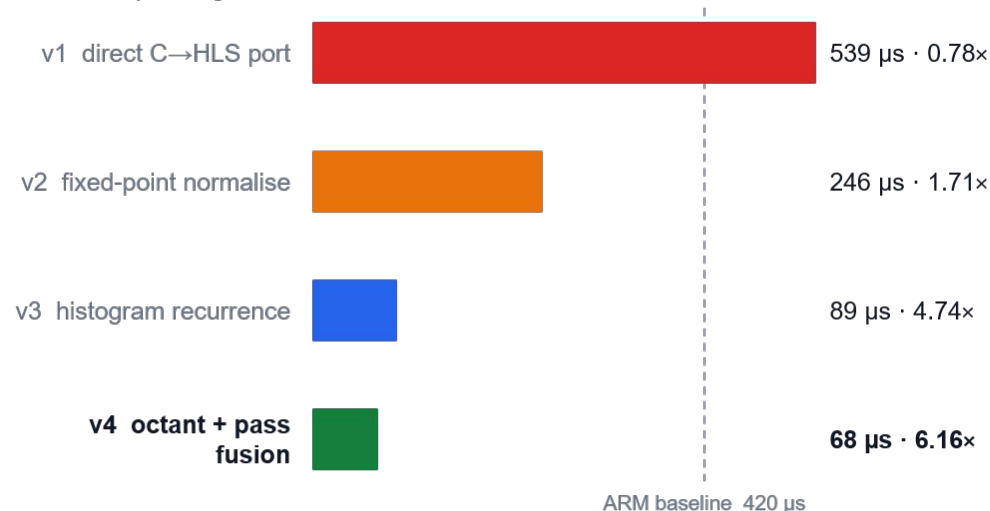
Design decision One kernel launch amortises control; the PL never waits on the PS between images.

System boundary: PetaLinux + XRT native host → Vitis HLS PL kernel (xclbin via xmutil loadapp).

From 0.78× to 6.16×: rebuilding the datapath

The first working HLS port ran slower than the ARM it replaced. Every stage had a floating-point operator sitting on a loop-carried path that could not close 5 ns.

Kernel time per image



The final change v4

Octant classifier. $\text{atan}(\text{dy}/\text{dx})$ replaced by comparisons of dy against $\text{dx} \cdot \tan \theta$. Same 8 bins, no divide, no transcendental 17,755 LUT freed.

Pass fusion. Variance folded into the gamma pass, normalisation folded into the gradients, 4 cells histogrammed per cycle. Three of five passes deleted.

Result board, 10,000 images

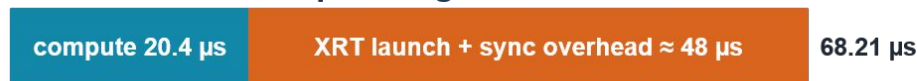
- ✓ 6.16× kernel · 5.56× end-to-end
- ✓ 9,998 / 10,000 within RMSE $1\text{e-}2$
- ✓ Timing closed at 3.65 ns of a 5 ns target
- ✓ LUT 61% → 48% while getting faster

Lesson Fixed point wasn't a precision trade it was what made pipelining physically possible at 5 ns.

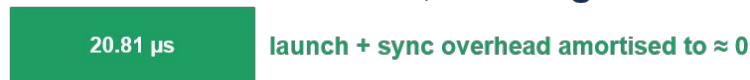
Optimisation I measure first: 6.16× → 20.2×

The board said 68.2 μ s per image. Synthesis said the compute was only 20.4 μ s. The other 48 μ s was XRT launch overhead paid 10,000 times.

v4 one XRT launch per image



v5 one launch for all 10,000 images



Board kernel time landed within 0.5% of the synthesis estimate once the per-launch cost was gone.

THE CHANGE

Batched kernel interface: `hog_compute(images*, features*, num_images)`. The kernel loops over images; the host packs one big buffer, syncs once each way, launches once. 16-wide copy lanes feed the auto-widened 512-bit AXI ports.

RESULT BOARD, 10,000 IMAGES

- ✓ 20.81 μ s / image — 20.16× kernel, 20.03× end-to-end
- ✓ Transfer overhead 9.8% → 0.6% of runtime
- ✓ Zero change to compute logic or resources (LUT 51%)

Lesson Profile the system, not just the kernel the biggest single win never touched the datapath.

Verified: csim signature unchanged · cosim (random_stall) pass · board n=50 then n=10,000 with reference cross-check.

Optimisation II inside the kernel: 20× → 33×

Two verified tiers on the two biggest cycle sinks and one honest detour that synthesis caught before it reached hardware.

TIER 1 · BLOCK NORMALISATION

1,491 → 385 cycles

Precompute 16 per-cell energies once (interior cells are shared by up to 4 overlapping blocks) instead of re-squaring 32 values per block; replace the 48-cycle fixed-point sqrt with a pipelined float `hls::sqrt` valid here because there is no recurrence.

kernel 15.3 μ s · $\approx 27\times$ · LUT 60%

THE DETOUR: REVERTED LUT 51% → 92%

First attempt ran 3 fixed-point sqrts in parallel via inlining. Vitis auto-pipelines small loops and pipelining fully unrolls every loop in the body, flattening a 48-stage, 98-bit recurrence into $\approx 45k$ LUTs of combinational logic, $\times 3$.

reverted same day · zero cost to schedule

TIER 2 · NORMALISE 2-WIDE 1,132 → 621 cycles

Process an even/odd pixel pair per clock with lane-split accumulators the reassociation is provably bit-exact in fixed point, so the csim signature is untouched. Deliberately 2-wide, not 4: wider buys nothing once dataflow makes gradients the ceiling.

kernel 12.7 μ s · $\approx 33\times$ · LUT 65%

Version	Change	Cycles / image	μ s / image	LUT	DSP
v5	batched interface	4,175	20.9	51%	4%
v6	3-wide sqrt (reverted)	3,063	15.3	92%	13%
v7	energy precompute + float sqrt	3,058	15.3	60%	13%
v8	2-wide normalise	2,547	12.7	65%	15%

Every row: csim max-diff signature unchanged at 0.00000310; cosim with random_stall passed on the combined v8 state.

Optimisation III dataflow: 33× → 81× → 135.5×

Stop finishing one image before starting the next then widen whichever stage sets the pace.

V9 · SIX-STAGE DATAFLOW PIPELINE

Per-image interval 2,547 → 1,034 cycles. Kernel dissolved into 6 concurrent stages with ping-pong channels: while image n computes gradients, $n+1$ normalises and $n+2$ loads. Throughput now equals the slowest stage, not the sum (Kastner ch. 8).

V10 · WIDEN THE PACING STAGE

Gradients 1,033 → 583 cycles. 2 pixels per clock. The 8 reads/cycle need a 6-bank partition of the gamma image — rows cyclic $\times 3$ ($h-1$, h , $h+1$ always in distinct banks) $\times$ columns cyclic $\times 2$. Interval drops to 622 cycles = 3.11 μ s per image.

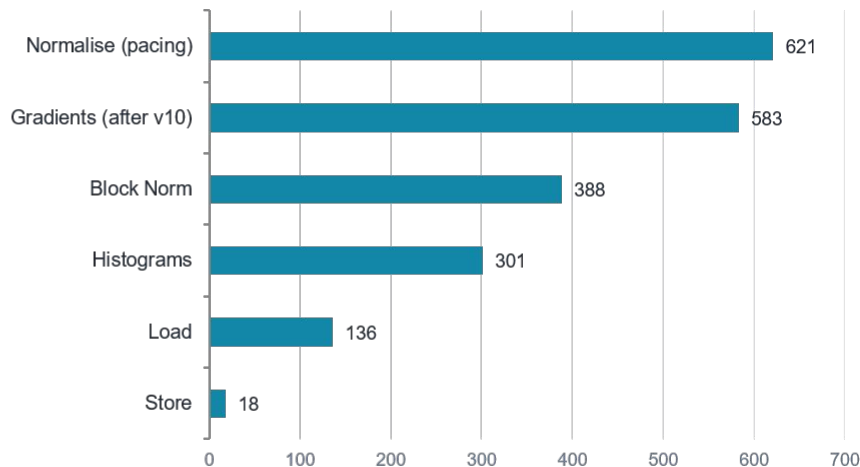
V9 ON BOARD

5.15 μ s · 81.5×

V10 ON BOARD

3.10 μ s · 135.5×

Final stage balance cycles per image (interval = slowest stage)



BRAM 14% → 40% — the honest price of ping-pong channels; LUT stays at 66%.

Dataflow verified in RTL cosim under random stalling: measured steady-state interval matched synthesis within 1%.

Results: method & numbers

Same 10,000 images through both implementations, on real hardware, cross-checked against two references every run.

BENCHMARKING METHOD

- 10,000 CIFAR-10 grayscale 32×32 BMPs identical set for PS and PL
- Timings exclude file loading and one warm-up launch
- Phases timed separately: H2D write · kernel · D2H read

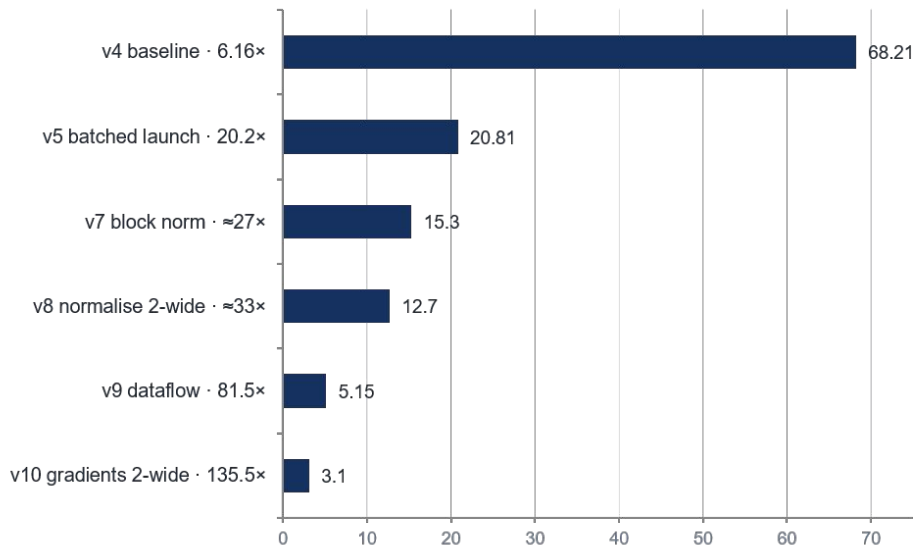
FINAL · BOARD-MEASURED · 10,000 IMAGES

135.50× kernel · **129.69×** end-to-end

SW 420 μ s → HW 3.10 μ s per image · 308,724 images/s · full batch in 32.4 ms

Resources: LUT 66% · BRAM 40% · DSP 17% · FF 19% · timing closed at 3.65 ns / 5 ns

The journey μ s per image on the board (lower is better)



Correctness constant across all six versions: $RMSE \approx 2.5 \times 10^{-5}$ vs reference; the same 2 known bin-boundary outliers every time.

v7/v8 μ s are synthesis-cycle equivalents verified in cosim; v4, v5, v9, v10 are board-measured runs of the full 10,000-image benchmark.

Discussions

Six aggressive changes, zero correctness regressions because failures were caught early and cheaply.

WHAT WORKED

- ❖ **Measuring the system first.**
The 48 μ s launch-overhead insight was worth 3.3 $\times$ alone before touching any compute.
- ❖ **Dataflow.** The single largest jump (33 $\times$ $\rightarrow$ 81 $\times$); throughput = slowest stage, then widen that stage.
- ❖ **Bit-exact refactor proofs.** A fixed csim signature made every regression instantly visible.

WHAT DIDN'T (AND WHY)

- ❖ **3-wide inlined fixed-point sqrt.**
HLS auto-pipeline fully unrolled a 48-stage recurrence $\rightarrow$ LUT 92%. Mechanism understood, reverted, replaced with float `hls::sqrt`.
- ❖ **Two stale-build incidents.** csim silently ran old source; build markers caught both before any wrong conclusion was drawn.

REJECTED

- ❖ **Line buffers / `hls::stream`.** The textbook fix for stencils but HoG needs the global mean and stddev before any gradient, forcing a full-frame barrier. Frame buffers are required either way (Kastner ch. 9 analysis).
- ❖ **Wider unrolls everywhere.** Past a balanced pipeline, each halving costs $\sim$ 12k LUTs for shrinking returns not worth the implementation risk.

Conclusion

6.16x → 135.5x, verified at every step.

Every item on our interim 'given more time' list was delivered and exceeded block norm optimised, outliers explained, and the speedup multiplied 22-fold with zero correctness regressions.

GIVEN MORE TIME...

- ❖ 4-wide normalise moves the ceiling to ≈ 590 cycles ($\approx 145\times$) for $\approx 12k$ LUTs
- ❖ Try a 250 MHz kernel clock the 3.65 ns timing estimate suggests real headroom
- ❖ Split normalise into sub-stages under dataflow for a deeper pipeline
- ❖ SVM classification stage on-fabric descriptors never leave the PL

WHY WE STOPPED AT 135.5x

- ❖ The pipeline is balanced: 621 / 583 / 388 / 301 cycles no cheap bottleneck left
- ❖ LUT at 66% the next halving risks Vivado implementation for a shrinking headline
- ❖ A deliberate engineering decision, argued from the numbers not a limitation

Thank you!

Any questions? · github.com/AbhiramYanamandra/Accelerated-HoG